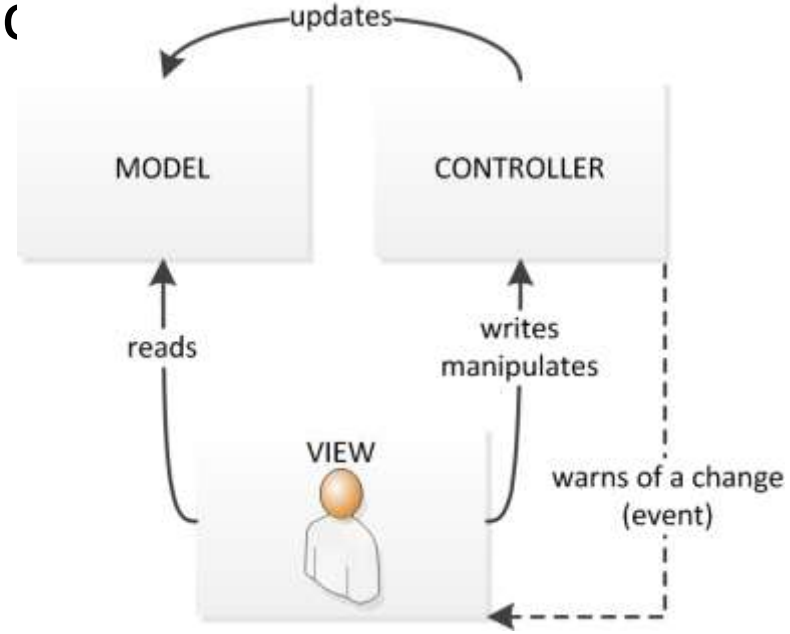
CSE470: Software Engineering

MVC Architecture

What is MVC Architecture

- MVC goes for Model-View-Controller Pattern
- Separates presentation and interaction from the data handling logic.
- It provides an extensible separation of concern (SOC) from presentation view t



Controller

- Handles all the user inputs through url
- The interaction to an application starts here by the user interactions – mouse click, key press etc
- Process http url requests (GET, POST, PUT, DELETE)
  - GET: for getting a data
  - POST: for posting / inserting a data
  - PUT: for updating a data
  - DELETE: for removing a data
- Communicates with both Model and View
- Contains all server side logic
- In the example – ProductController, UserController, AccountController etc.

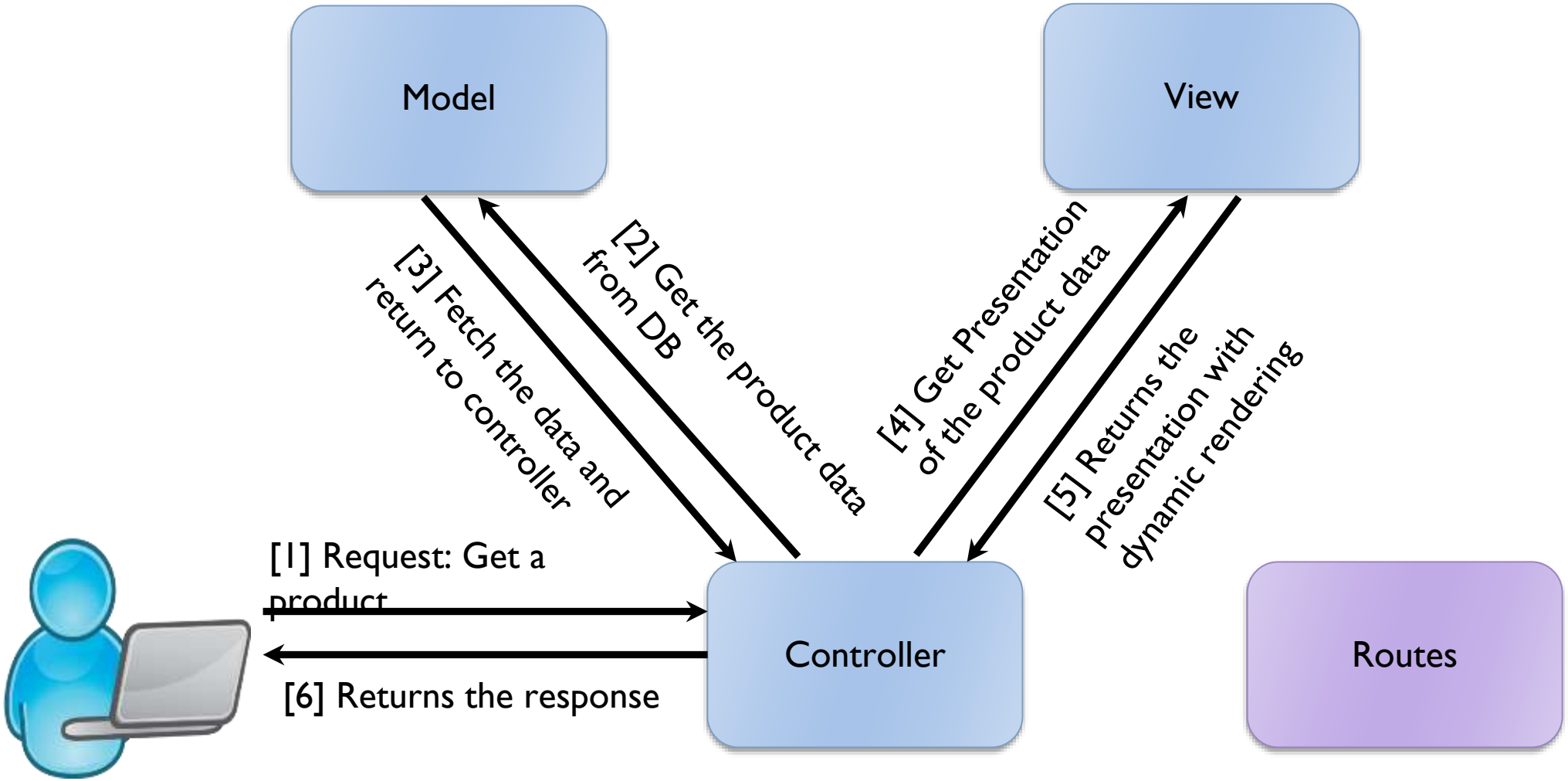
Model

- It refers to the Data Related Logic
- Interaction with database (such as SELECT, INSERT, UPDATE, DELETE)
- It communicates with the controllers
- Can sometimes update or collaborate with the view (Depends on framework)
- In the example – Product, User, Transaction, Cart etc are model classes.

View

- What the end users see (UI)
- Usually Consists of html/css
- Communicates with the controller
- While coding are passed as dynamic values from the controller
- A controller can have multiple associated views.
- In the example – product.html, user.html to view a html file

MVC Steps


Model
View
[2] Get the product data from DB
[3] Fetch the data and return to controller
[4] Get Presentation of the product data
[5] Returns the presentation with dynamic rendering
[1] Request: Get a product
[6] Returns the response
Controller
Routes

Routes

- Are urls to access a resource kept under App_Start folder
- General structure - http://domainName/{controller}/{action}/{id}
- http://YourApp.com/Users/Profile/25

```
public class RouteConfig
{
    public static void RegisterRoutes(RouteCollection routes)
    {
        routes.IgnoreRoute("{resource}.axd/{*pathInfo}");

        routes.MapRoute(
            name: "Default",
            url: "{controller}/{action}/{id}",
            defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
        );
    }
}
```

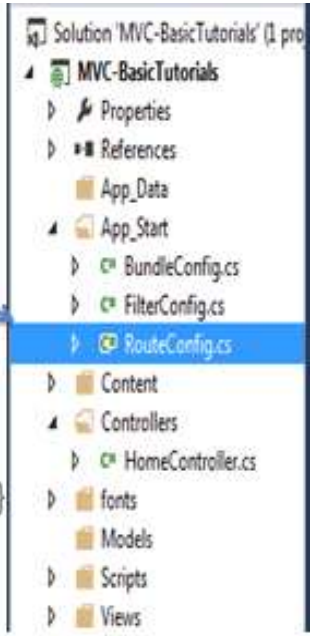
Route to ignore

Route name

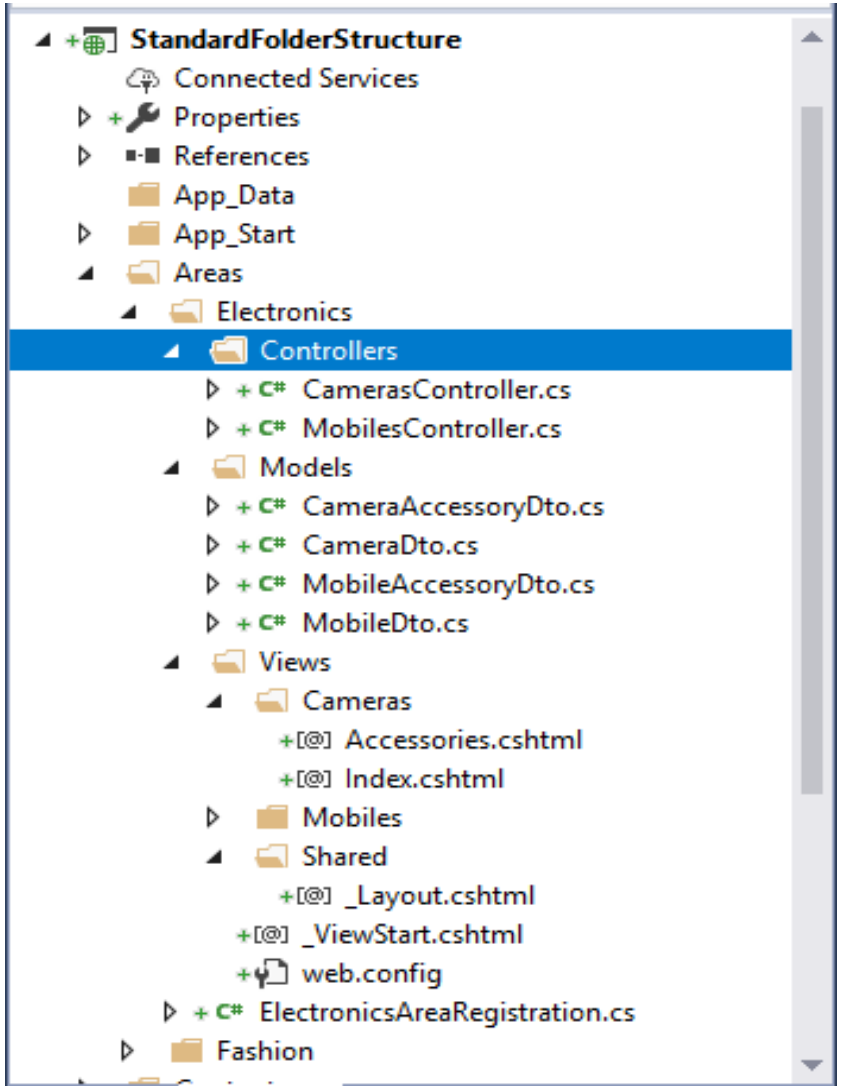
URL Pattern

Defaults for Route

RouteConfig.cs



MVC Folder Structure



MVC Code Structure

```
http://yourapp.com/users/profile/1

/routes
    users/profile/:id = Users.getProfile(id)

/controllers
    class Users{
        function getProfile(id){
            profile = this.UserModel.getProfile(id)

            renderView('users/profile', profile)
    }

/models
    Class UserModel{
        function getProfile(id){
            data = this.db.get('SELECT * FROM users WHERE id = id')
            return data;
    }

/views
    /users
        /profile
        <h1>{{profile.name}}</h1>
        <ul>
            <li>Email: {{profile.email}}</li>
            <li>Phone: {{profile.phone}}</li>
```

MVC Architecture

Request Flow — Student Record Lookup

Browser → Router → Controller → Model → Database | Database → Model → Controller → View → Browser

Step 1/10

Browser sends GET /students/2021-CSE-047

Request path →

Browser

HTTP client

Student lookup

2021-CSE-047 | Submit

User submits → browser fires HTTP GET

Controller

StudentController

View

student_detail.html

Database

MySQL / ORM

Model

StudentModel

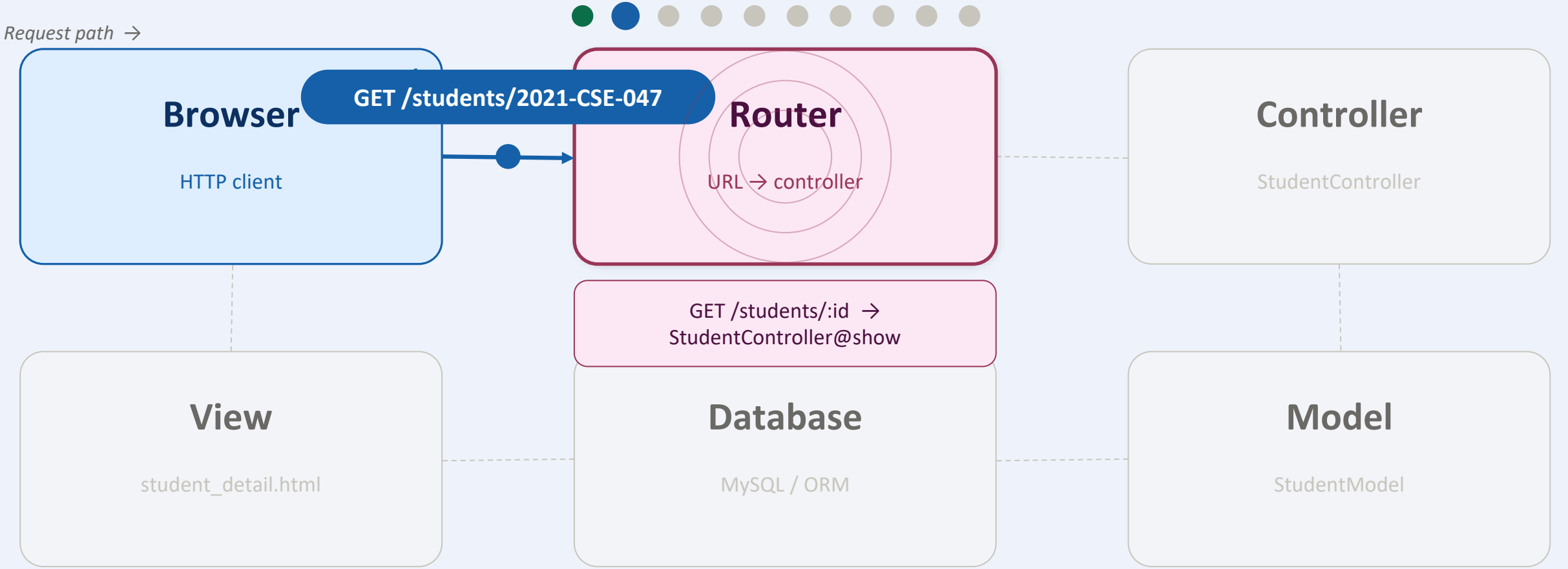
← Response path

Network log

→ **REQUEST** GET /students/2021-CSE-047 — user submits form, browser fires HTTP request

Step 2/10

Router matches URL to Controller



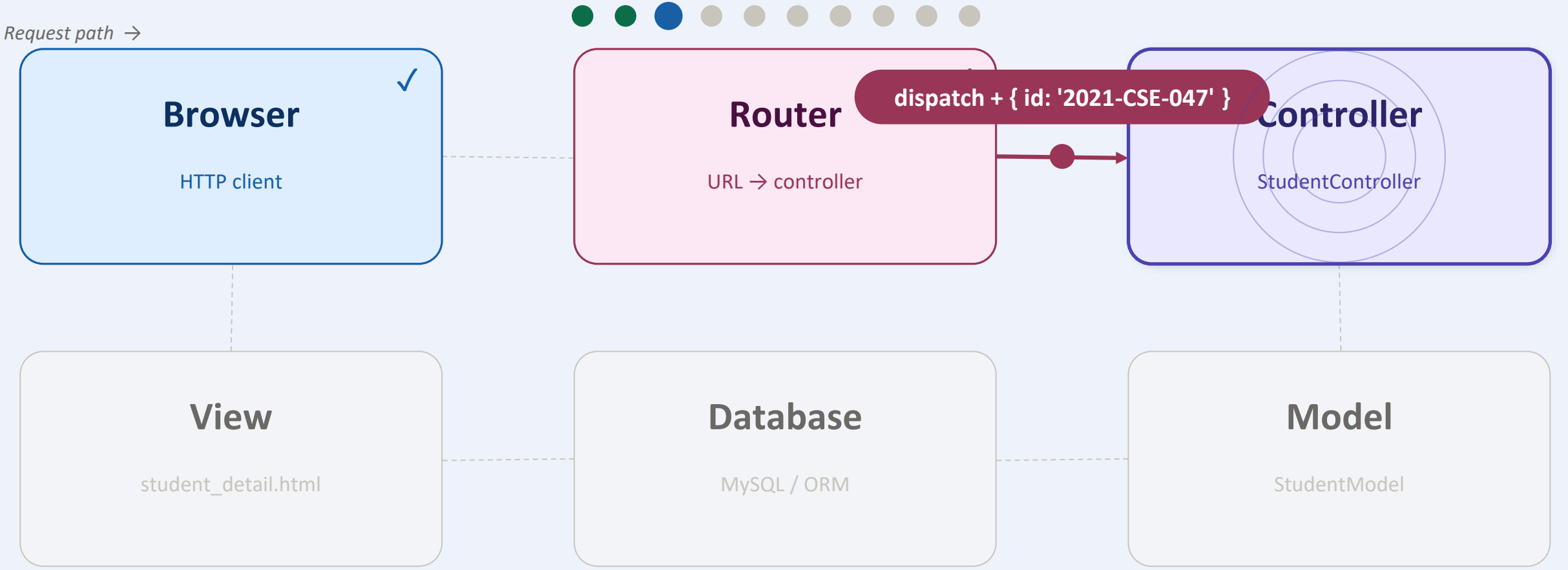
Network log

→ **REQUEST** GET /students/2021-CSE-047 received by Router

⟳ **ROUTER** Pattern match: GET /students/:id → StudentController@show

Step 3/10

Router dispatches request to Controller



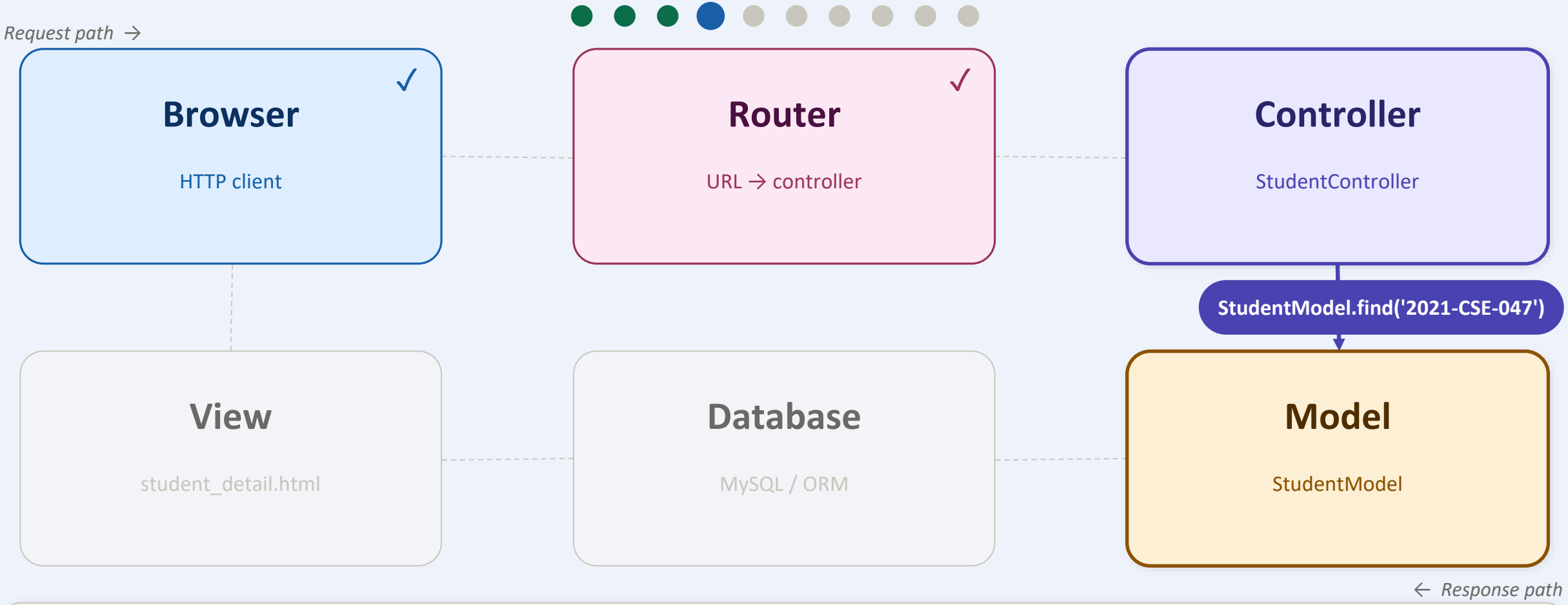
Network log

ROUTER Route matched — dispatching to StudentController with bound params

CONTROLLER StudentController.show(id='2021-CSE-047') invoked

Step 4/10

Controller calls Model with bound ID



Network log

CONTROLLER $id = '2021-CSE-047' bound from route param

CONTROLLER StudentModel.find($id) called — passing bound value to Model

Step 5/10

Model sends SQL query to Database

Request path →

Browser ✓

HTTP client

Router ✓

URL → controller

Controller ✓

StudentController

View

student_detail.html

Database

MySQL / ORM

Model

StudentModel

SELECT * WHERE id='2021-CSE-047'

← Response path

Network log

◈ **MODEL** ORM builds query: SELECT * FROM students WHERE id = '2021-CSE-047'

⬡ **DB** SQL query dispatched to database over internal connection

Step 6/10

Database returns result row to Model

Request path →

Browser ✓

HTTP client

Router ✓

URL → controller

Controller ✓

StudentController

View

student_detail.html

Database

MySQL / ORM

{ student row }

Model

StudentModel

← Response path

Network log

⬡ **DB** Query executed in 4ms — 1 row found (Rafiul Hasan)

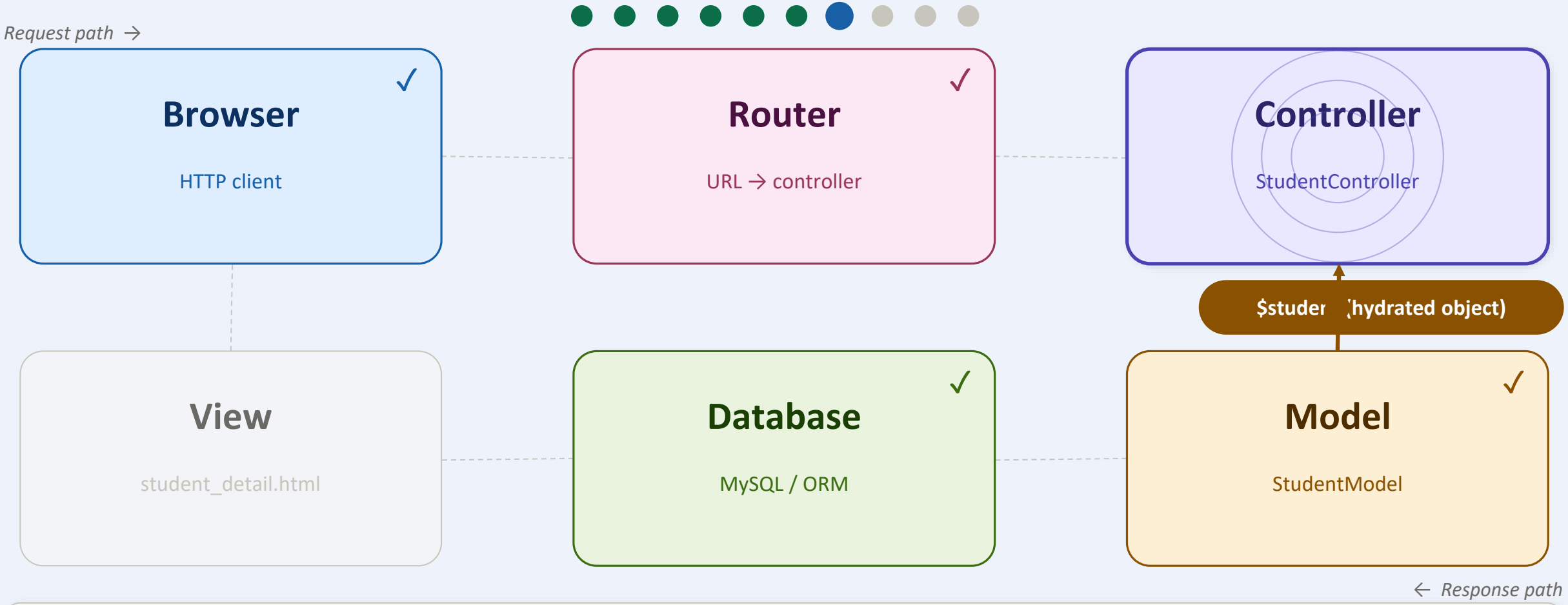
⬡ **DB** Raw row { id, name, dept, cgpa, semester, status } returned to Model

Result: 1 row found

name:	Rafiul Hasan
cgpa:	3.72
dept:	CSE
semester:	8th

Step 7/10

Model returns bound $student object to Controller



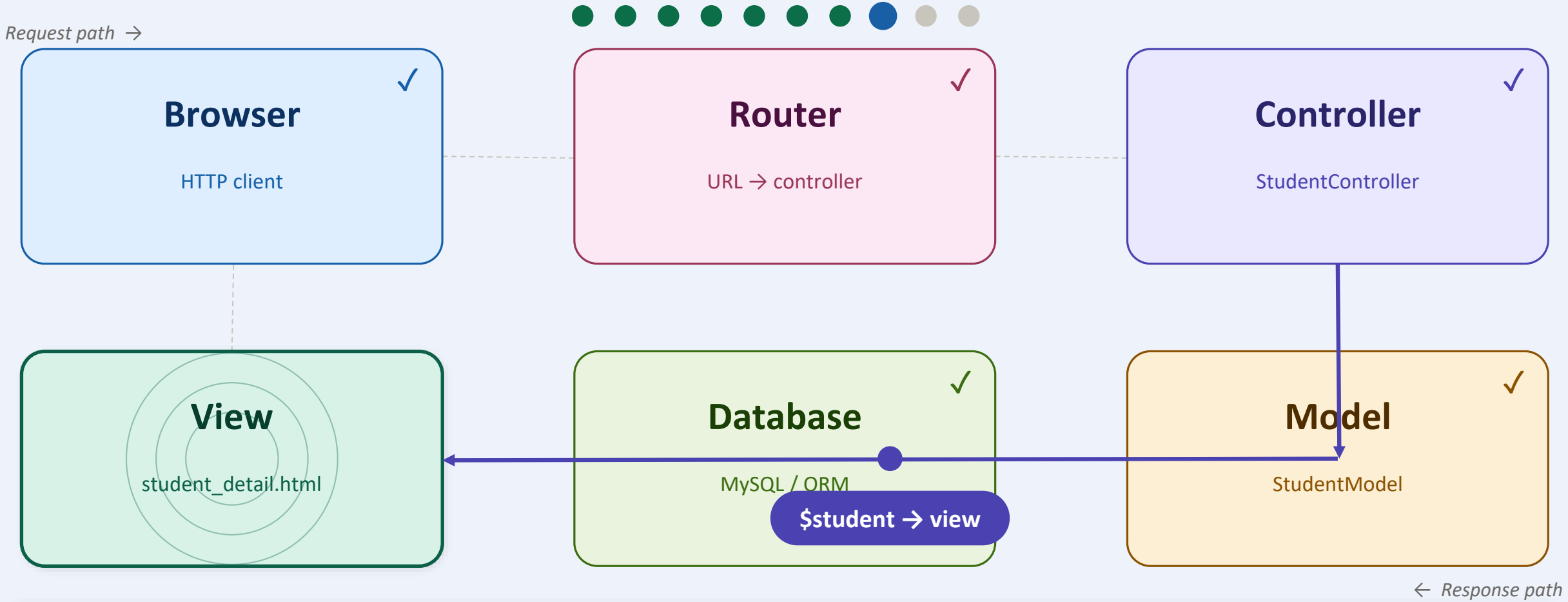
Network log

◈ **MODEL** Row mapped → Student object: { name, dept, cgpa, semester, status }

CONTROLLER $student hydrated object received back in Controller

Step 8/10

Controller passes $student to View



Network log

CONTROLLER $student object ready — calling return view('student_detail', $student)

VIEW View receives $student data — beginning template compilation

Step 9/10

View renders student_detail.html template

Request path →

Browser ✓

HTTP client

Router ✓

URL → controller

Controller ✓

StudentController

View

student_detail.html

Database ✓

MySQL / ORM

Model ✓

StudentModel

{{ $student->name }} → Rafiul Hasan

← Response path

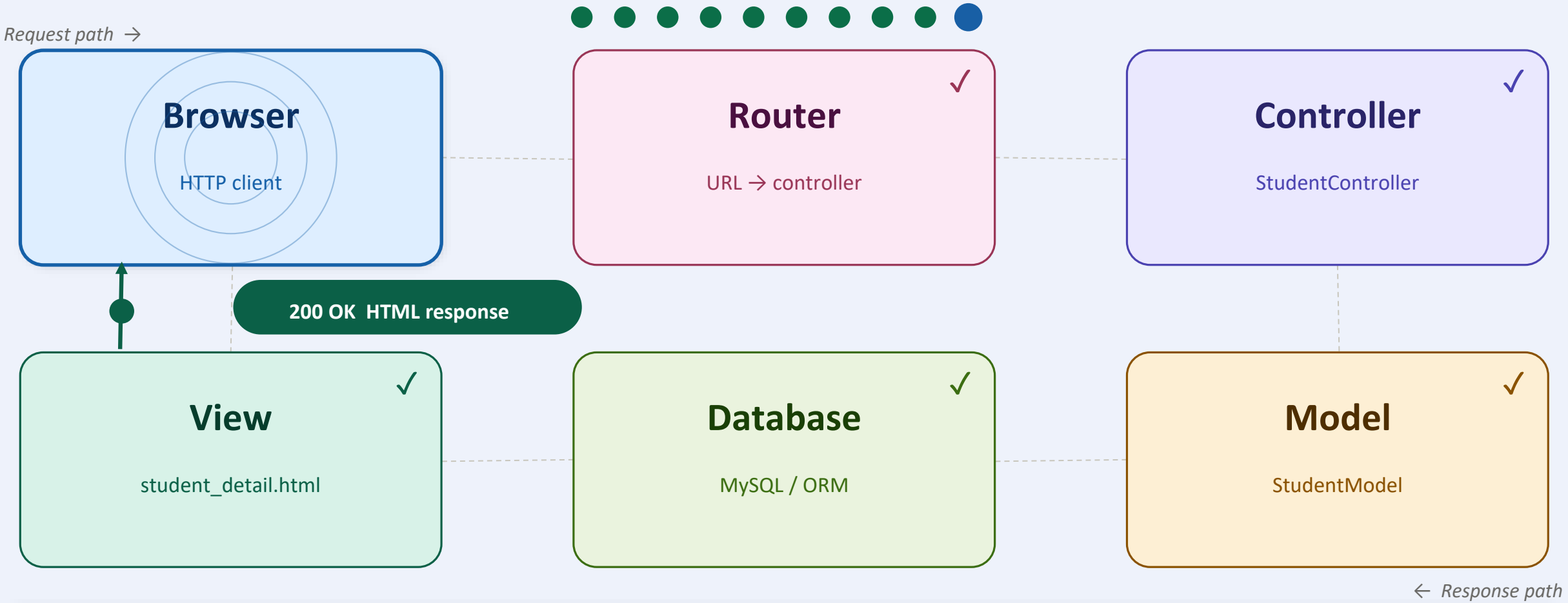
Network log

☐ **VIEW** Blade/template engine binding {{ $student->name }}, {{ $student->cgpa }}…

☐ **VIEW** 200 OK response compiled — HTML ready to send back through Router to Browser

Step 10/10

200 OK HTML response delivered to Browser



Network log

☐ **VIEW** View → Router → Browser: 200 OK with compiled student_detail.html

✓ **RESPONSE** Browser renders page — MVC cycle complete in ~180ms

MVC cycle complete — student page rendered

RH

Rafiul Hasan

2021-CSE-047 · rendered via student_detail.html

CGPA	SEMESTER
3.72 / 4.00	8th

DEPARTMENT	STATUS
CSE	Active

Network log

✓ **RESPONSE** 200 OK — MVC cycle complete — student_detail.html rendered in browser

✓ **RESPONSE** Full path: Browser → Router → Controller → Model → Database → Model → Controller → View → Browser

MVC request lifecycle

1. Browser fires GET /students/2021-CSE-047
2. Router pattern-matches URL to controller
3. Router dispatches + bound { id } to Controller
4. Controller calls Model.find($id)
5. Model sends SQL query to Database
6. Database returns raw row to Model
7. Model returns hydrated $student to Controller
8. Controller passes $student to View
9. View renders HTML template
10. 200 OK HTML delivered to Browser

Entire MVC cycle completes in under 200ms

When to use MVC

- Large or Complex Applications
- Frequent UI or Logic Updates

Note: Your final project MUST follow MVC Architecture

Advantages-Disadvantages

Advantages:

Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them. Other advantages are: parallel working, improved testability, code reusability

Disadvantages:

Can involve additional code and code complexity when the data model and interactions are simple.

JavaScript-based MVC Web App: Click here